Advanced Programming Java

Sessional 2 - Detailed Solutions

Q-1. Explain JDBC architecture.

Introduction to JDBC:

JDBC (Java Database Connectivity) is a Java API that manages connecting to a database, issuing queries and commands, and handling result sets obtained from the database. It allows Java applications to interact with relational databases (like MySQL, Oracle, PostgreSQL) in a standardized way.

JDBC Architecture:

The JDBC architecture consists of two main processing layers:

1. **JDBC API Layer:** This provides the application-to-JDBC Manager connection. It is used by the Java application developers to connect to the database.
2. **JDBC Driver API Layer:** This supports the JDBC Manager-to-Driver Connection. It acts as an interface between the JDBC API and the actual database.

Components of JDBC Architecture:

- **Application:** The Java program that uses the JDBC API to communicate with the database.
- **JDBC API:** Contains interfaces and classes (like `Connection`, `Statement`, `PreparedStatement`, `ResultSet`) required to communicate with the database.
- **DriverManager:** A class that manages a list of database drivers. It matches connection requests from the Java application with the proper database driver using communication subprotocols.
- **JDBC Drivers:** These are software components that enable a Java application to interact with the database. There are four types of JDBC drivers:
    1. **Type 1 (JDBC-ODBC Bridge Driver):** Translates JDBC API calls into ODBC calls. Now obsolete.
    2. **Type 2 (Native-API Driver):** Converts JDBC calls into native C/C++ API calls of the specific database.
    3. **Type 3 (Network Protocol Driver):** Translates JDBC calls into a middleware-specific protocol, which is then translated by a middleware server into a database-specific protocol.
    4. **Type 4 (Thin Driver):** Converts JDBC calls directly into the vendor-specific database protocol. It is written completely in Java and is the most widely used driver today.

Q-2. Explain servlet along with its life cycle.

What is a Servlet?

A Servlet is a server-side Java program or technology used to create dynamic web applications. It extends the capabilities of a server and handles incoming requests from the client (usually a web browser), processes them (often by interacting with a database), and sends back a response (usually in the form of HTML). Servlets are robust, scalable, and faster than older CGI (Common Gateway Interface) scripts.

Servlet Life Cycle:

The life cycle of a servlet is managed by the Servlet Container (like Apache Tomcat). It consists of four main phases:

1. **Loading and Instantiation:**
   When the server starts or when the first request for the servlet is received, the Web Container loads the Servlet class and creates an instance (object) of it. This happens only once during the servlet's lifespan.

2. **Initialization (`init` method):**
   After instantiation, the container calls the `init()` method.

   - This method is used for one-time initialization setups, such as reading configuration files or opening database connections.
   - It is called **only once** in the entire life cycle.

```
public void init(ServletConfig config) throws ServletException {
    // Initialization code here
}
```

3. **Request Handling (`service` method):**
   This is the core phase. Whenever a client makes a request to the servlet, the container spawns a new thread and calls the `service()` method.

   - The `service()` method checks the HTTP request type (GET, POST, PUT, DELETE, etc.) and dispatches the request to the appropriate method like `doGet()`, `doPost()`, etc.
   - This method is called **multiple times**, once for every client request.

```
public void service(ServletRequest request, ServletResponse response)
    throws ServletException, IOException {
    // Request handling code here
}
```

4. **Destruction (`destroy` method):**
   When the container decides to unload the servlet (e.g., server shutdown or memory constraints), it calls the `destroy()` method.
   - This method gives the servlet a chance to close database connections, halt background threads, and perform cleanup activities.
   - It is called **only once** at the end of the life cycle.

```
public void destroy() {
    // Cleanup code here
}
```

Q-3. Differentiate between method overloading and method overriding with examples.

Method Overloading:

Method overloading occurs when two or more methods in the **same class** have the same name but different parameters (different number of parameters, different types of parameters, or both). It is an example of compile-time (static) polymorphism.

Method Overriding:

Method overriding occurs when a **subclass (child class)** provides a specific implementation for a method that is already defined in its superclass (parent class). The method signature must be exactly the same. It is an example of run-time (dynamic) polymorphism.

Key Differences:

Feature	Method Overloading	Method Overriding
Location	Happens within the same class.	Happens between two classes (Superclass and Subclass) that have an inheritance relationship.
Method Name	Must be the same.	Must be the same.
Parameters/ Signature	Must be different (type, number, or order).	Must be exactly the same.
Return Type	Can be the same or different.	Must be the same or a covariant type.
Polymorphism	Compile-time (Static binding).	Run-time (Dynamic binding).
Keywords	No specific keyword required.	Often uses the `@Override` annotation for clarity.

Example of Method Overloading:

```
class MathOperations {
    // Method with two int parameters
    int add(int a, int b) {
        return a + b;
    }
    // Overloaded method with three int parameters
    int add(int a, int b, int c) {
        return a + b + c;
    }
}
```

Example of Method Overriding:

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    // Overriding the sound method of the parent class
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}
```

Q-4. Explain object-oriented concepts in Java including class, object, constructors, encapsulation, inheritance, method overloading, and method overriding with examples.

Java is heavily based on Object-Oriented Programming (OOP) paradigms. Here are the core concepts:

1. Class:

A class is a blueprint or a template for creating objects. It defines the properties (attributes/variables) and behaviors (methods) that the objects created from it will have.

```
class Car {
    String color; // property
    void drive() { // behavior
        System.out.println("Car is driving");
    }
}
```

2. Object:

An object is an instance of a class. It is a real-world entity that has a state and behavior defined by its class.

```
Car myCar = new Car(); // myCar is an object of class Car
```

3. Constructors:

A constructor is a special block of code that is called when an object is instantiated. It has the same name as the class and no return type. Its primary purpose is to initialize the object's state.

```
class Student {
    String name;
    // Constructor
    Student(String n) {
        name = n;
    }
}
```

4. Encapsulation:

Encapsulation is the mechanism of wrapping the data (variables) and code acting on the data (methods) together as a single unit. In Java, this is achieved by declaring the variables of a class as `private` and providing `public` getter and setter methods to modify and view the variables.

```
class Person {
    private int age; // Data hiding

    public int getAge() { return age; }
    public void setAge(int a) { age = a; }
}
```

5. Inheritance:

Inheritance is a mechanism where a new class (subclass/child) acquires the properties and behaviors of an existing class (superclass/parent). It promotes code reusability and establishes an IS-A relationship. It is achieved using the `extends` keyword.

```
class Vehicle {
    int speed = 50;
}
class Bike extends Vehicle {
    // Bike inherits 'speed' from Vehicle
}
```

6. Method Overloading & 7. Method Overriding:

(Please refer to the detailed explanation and examples provided in Q-3 above, as the concepts are identical.)

Q-5. Explain loops, conditional statements, and arrays in Java.

1. Conditional Statements (Decision Making):

Conditional statements allow a program to execute specific blocks of code based on certain conditions (evaluating to `true` or `false`).

- **`if` statement:** Executes a block of code if a condition is true.
- **`if-else` statement:** Executes one block if true, and another if false.
- **`else-if` ladder:** Used to test multiple conditions in sequence.
- **`switch` statement:** Selects one of many code blocks to be executed based on the value of a variable.

Example:

```
int marks = 85;
if (marks >= 90) {
    System.out.println("Grade A");
} else if (marks >= 80) {
    System.out.println("Grade B");
} else {
    System.out.println("Grade C");
}
```

2. Loops (Iteration Statements):

Loops are used to execute a set of instructions repeatedly as long as a specified condition remains true.

- **`for` loop:** Used when the exact number of iterations is known beforehand. It combines initialization, condition, and increment/decrement in one line.
- **`while` loop:** Used when the number of iterations is not known. It checks the condition *before* executing the loop body.
- **`do-while` loop:** Similar to the `while` loop, but it checks the condition *after* executing the loop body. This guarantees the loop runs at least once.

Example:

```
for (int i = 1; i <= 5; i++) {
    System.out.println("Iteration: " + i);
}
```

3. Arrays:

An array is a data structure used to store a fixed-size sequential collection of elements of the same data type. In Java, arrays are dynamically allocated objects.

- **Key properties:** Elements are accessed using an index (starting from 0). Once the size of an array is defined, it cannot be changed.

Declaration and Initialization:

```
// Declaration and memory allocation for 5 integers
int[] numbers = new int[5];

// Assigning values
numbers[0] = 10;
numbers[1] = 20;

// Declaration and initialization in one line
String[] fruits = {"Apple", "Banana", "Orange"};

// Accessing an element
System.out.println(fruits[1]); // Outputs: Banana
```